Number-theoretic algorithms solutions

31-E01 Euclid's certificate and input length

The divisions are $391 = 299 + 92$, $299 = 3 \cdot 92 + 23$, and $92 = 4 \cdot 23 + 0$. Thus the gcd is 23. Back substitution gives $23 = 299 - 3(391 - 299) = 4 \cdot 299 - 3 \cdot 391$. Coefficients in input order are $(-3, 4)$, verified by $-1173 + 1196 = 23$. Since the gcd is not 1, 299 has no inverse modulo 391. An equation $299u - 391v = 1$ would have a left side divisible by 23, a contradiction.

For $a \geq b > 0$, let $r = a \bmod b$. If $r \leq b/2$, the second argument already halves. Otherwise the next remainder is $b - r < b/2$. Thus β-bit inputs take $O(\beta)$ divisions. A long division can cost $O(\beta^2)$ bit operations, giving the safe bound $O(\beta^3)$. Large-integer division is not one bit operation. The absent inverse reflects the necessary coprimality condition.

31-E02 All modular linear solutions

Here $\gcd(24, 60) = 12$, which divides 36. Divide by 12 to get $2x \equiv 3 \pmod 5$. Since $2^{-1} = 3$, $x \equiv 9 \equiv 4 \pmod 5$. The twelve representatives modulo 60 are 4,9,14,19,24,29,34,39,44,49,54,59. An increase of 5 increases the product by 120, a multiple of 60. Any solution satisfies $5 \mid 2(x - x_0)$, and cancellation gives $5 \mid x - x_0$, proving completeness.

Since $12 \nmid 35$, the second equation has no solution. The equation $0x \equiv 0 \pmod 9$ holds for $x = 0, \ldots, 8$, compressed as `(x0=0, step=1, count=9)`. Generally there are n representatives. If n has β bits, enumeration can be exponentially large in β, while compression needs $O(\beta)$ bits. There is no inverse of 24 modulo 60 to multiply by.

31-E03 CRT construction and compatibility

We have $M = 4 \cdot 9 \cdot 5 = 180$, $M_i = (45, 20, 36)$, and inverses modulo 4,9,5 of $(1, 5, 1)$. Selectors are $(45, 100, 36)$. Reconstruction gives $x = 45 + 2 \cdot 100 + 3 \cdot 36 = 353 \equiv 173 \pmod{180}$. Its residues are 1,2,3. Pairwise coprimality proves uniqueness modulo 180.

For 8 and 12, $g = 4$ divides $7 - 3 = 4$. Set $x = 3 + 8k$, giving $8k \equiv 4 \pmod{12}$, or $2k \equiv 1 \pmod 3$. Thus $k \equiv 2$, $x \equiv 19 \pmod{24}$, with period $\operatorname{lcm}(8, 12) = 24$. Modulo 96 the representatives are 19,43,67,91. Replacing 7 with 6 gives difference 3, not divisible by 4, violating a necessary condition for any common solution. The product modulus is not the least period in the noncoprime case.

Number-theoretic algorithms solutions (continued)

31-E04 Orders, primitive roots, and different groups

For $k = 0, \ldots, 10$, the powers $2^k \pmod{11}$ are 1,2,4,8,5,10,9,7,3,6,1. The first positive return to 1 is at 10, proving order 10. The logarithm of 7 is $7 \pmod{10}$.

Every member of $U(8) = \{1, 3, 5, 7\}$ squares to 1. All orders are at most 2, so no element generates all four members. Inverses do not imply a generator.

Modulo 121, $2^{10} = 1024 \equiv 56 = 1 + 5 \cdot 11 \neq 1$. Reduction modulo 11 makes the order divisible by 10. It also divides $\varphi(121) = 110$, so the only possibilities are 10 and 110. The calculation excludes 10. Also $(1 + 55)^{11} \equiv 1 \pmod{121}$, consistent with Euler's bound. Base 2 itself lifts, so changing to 13 is unnecessary. The square-level check is essential.

31-E05 Left-to-right and right-to-left powers

Start at $r = 1$. Bits 1,0,0,1,1 produce prefixes 1,2,4,9,19. The first bit squares 1 and multiplies by 7 to give 7. The second squares to $7^2 \equiv 3$. The third squares to 9. The fourth squares to $9^2 \equiv 12$, then multiplies by 7 to give 15. The fifth squares to $15^2 \equiv 18$, then multiplies by 7 to give 11. Thus $7^{19} \equiv 11 \pmod{23}$.

For right-to-left scanning, initialize $r = 1 \bmod n, a = A \bmod n, b = B$. While $b > 0$, if b is odd set $r \leftarrow ra \bmod n$, then set $a \leftarrow a^2 \bmod n, b \leftarrow \lfloor b/2 \rfloor$. The invariant is $ra^b \equiv A^B$. For $b = 2q$, the new product is $r(a^2)^q = ra^b$. For $b = 2q + 1$, it is $(ra)(a^2)^q = ra^b$. The exponent halves until zero, when r is the answer.

Exponent zero performs no iterations and returns $1 \bmod n$. For $n = 1$, it returns 0. No inverse is used, so noncoprime cases such as $A = 6, n = 15$ are valid. Negative exponents require a separate inverse check $\gcd(A, n) = 1$. A negative exponent passed to the halving loop can prevent termination. This scan tracks an unprocessed exponent, not an already-read prefix.

31-E06 Nonunit RSA and malleability

We have $n = 91, \varphi = 72$, and $1 = 29 \cdot 5 - 2 \cdot 72$, so $d = 29$. For $m = 7$, $7^2 \equiv 49, 7^4 \equiv 35, 7^5 \equiv 35 \cdot 7 \equiv 63 \pmod{91}$.

Decryption uses $63^2 \equiv 56, 63^4 \equiv 42, 63^8 \equiv 35, 63^{16} \equiv 42$. Since $29 = 16 + 8 + 4 + 1$, successive multiplications give $42 \cdot 35 \equiv 14$, $14 \cdot 42 \equiv 42$, and $42 \cdot 63 \equiv 7$.

Although $\gcd(7, 91) = 7$, modulo 7 the message and final power are both zero. Modulo 13, 7 is a unit and $ed = 145 \equiv 1 \pmod{12}$, so Fermat gives $7^{145} \equiv 7$. CRT proves recovery modulo 91.

Number-theoretic algorithms solutions (continued)

Since 2 is a unit, $c' = c2^5 \pmod{91}$ equals $E(2m \bmod 91)$, by $m^5 2^5 = (2m)^5$. Here $c' = 63 \cdot 32 \equiv 14$, decrypting to 14. This proves textbook RSA is malleable, not secure. Knowing factors yields d, but this does not prove equivalence of RSA inversion and factoring. The public key is $(91, 5)$ and private exponent is 29. Correctness, factoring hardness, and scheme security are separate claims.

31-E07 Count every strong liar

Factor $65 = 5 \cdot 13$ and $64 = 2^6 \cdot 1$, so $s = 6, d = 1$. The component decompositions are $5 - 1 = 2^2 \cdot 1$ and $13 - 1 = 2^2 \cdot 3$. Thus $t = (2, 2), u = (1, 3), D = (1, 1), \tau = 2, r = 2$. The formula gives $L = (1 + 1 + 4) \cdot 1 = 6$.

Initial $a^d = 1$ gives only 1. Level $j = 0$ gives -1, or 64. Level $j = 1$ requires $a^2 = -1 \pmod{65}$. Roots modulo 5 are 2,3 and modulo 13 are 5,8. The four CRT combinations give 8,18,47,57, each squaring to 64. They differ from 1,64, and the count proves completeness. The passing set is $\{1, 8, 18, 47, 57, 64\}$.

The actual domain $[2, 63]$ has 62 bases and four liars, giving $4/62 = 2/31 < 1/4$. Base 2 has chain 2,4,16,61,16,61 and never reaches 64. It is a compositeness witness. The next square is not 1, so it also fails Fermat. Roots can be combined independently only when -1 occurs at the same level.

31-E08 Random testing and posterior probability

Since $4^{14} = 268435456 < 10^9$ and $4^{15} = 1073741824 > 10^9$, at least 15 rounds are required. Let $\epsilon = 4^{-15} = 1/1073741824$. With prior $\pi = 1/100$, Bayes gives

$$\frac{99\epsilon}{1 + 99\epsilon} = \frac{99}{1073741824 + 99} \approx 9.22 \cdot 10^{-8}.$$

This exceeds the fixed-composite acceptance bound because the prior odds favor composites 99 to 1. The prior is a premise supplied by the question.

Repeating base 2 repeats the same event, so probabilities cannot be multiplied. For $2047 = 23 \cdot 89$, $2^{11} = 2048 \equiv 1$, and $d = 1023 = 11 \cdot 93$ gives $2^d = 1$. Every such round passes, however often repeated. The fifteen-round guarantee requires independent uniform bases. Posterior compositeness and fixed-input acceptance have different denominators.

Number-theoretic algorithms solutions (continued)

31-E09 Pollard rho component collisions

For 91, the first Floyd round gives $x = f(2) = 5, y = f(f(2)) = f(5) = 26$. The difference is 21, and $\gcd(21, 91) = 7$. The residues agree modulo 7 but are 5 and 0 modulo 13, giving proper factor 7 and cofactor 13.

For 15 with $f = x^2 + 2$, round 1 gives $x = 2, y = 6$, difference 4, gcd 1. Round 2 gives $x = f(2) = 6, y = f(f(6)) = f(8) = 6$. The difference is zero, and $\gcd(0, 15) = 15$. This gives the whole input rather than a proper divisor. Restart or report no factor found in this attempt.

A finite functional graph has tail length μ and cycle length λ. In round k, the pointers are at x_k, x_{2k}. For $k \geq \mu$, both are on the cycle, separated by $k \bmod \lambda$. Within λ more rounds, $k \equiv 0 \pmod{\lambda}$, so they meet. This proves $O(\mu + \lambda)$ rounds and a constant number of stored values.

The factor implication is deterministic. Estimating $\mu + \lambda = O(\sqrt{p})$ needs the additional assumption of approximately random-mapping behavior modulo p. That assumption gives no worst-case guarantee for a fixed polynomial. Success requires $1 < d < n$. Budget exhaustion does not establish primality.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Number-theoretic algorithms solutions (continued)

Verification code appendix: number_theory_checks.py

```
"""Check the number-theory proofs on exact small instances."""

from math import gcd


def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    q = a // b
    return g, y1, x1 - q * y1


def modexp(base, exponent, modulus):
    assert exponent >= 0 and modulus > 0
    value = 1 % modulus
    prefix = 0
    for bit in bin(exponent)[2:]:
        value = value * value % modulus
        prefix *= 2
        if bit == "1":
            value = value * base % modulus
            prefix += 1
        assert value == pow(base, prefix, modulus)
    return value


def strong_liars(n):
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2
        s += 1
    liars = []
    for base in range(2, n - 1):
        value = pow(base, d, n)
        if value in (1, n - 1):
            liars.append(base)
            continue
        for _ in range(s - 1):
            value = value * value % n
            if value == n - 1:
                liars.append(base)
                break
    return liars
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Number-theoretic algorithms solutions (continued)

```
def rsa_round_trip():
    p, q = 5, 11
    n, phi = p * q, (p - 1) * (q - 1)
    e, d = 3, 27
    assert e * d % phi == 1
    for message in range(n):
        cipher = pow(message, e, n)
        assert pow(cipher, d, n) == message


def run():
    g, x, y = egcd(30, 18)
    assert g == gcd(30, 18) == 6
    assert 30 * x + 18 * y == 6
    assert [(value, (3 * value - 6) % 12) for value in (2, 6, 10)] == [
        (2, 0),
        (6, 0),
        (10, 0),
    ]
    assert modexp(5, 13, 11) == 4
    assert modexp(5, 29, 11) == 9
    assert modexp(5, 0, 11) == 1
    rsa_round_trip()

    composite_cases = 0
    for n in range(9, 501, 2):
        if all(n % divisor for divisor in range(3, int(n**0.5) + 1, 2)):
            continue
        liars = strong_liars(n)
        assert len(liars) * 4 <= n - 1, (n, liars)
        composite_cases += 1
    print(
        "PASS gcd=6 modexp=4,9,1 rsa_residues=55 "
        f"strong_liar_bound_cases={composite_cases}"
    )


if __name__ == "__main__":
    run()
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Number-theoretic algorithms solutions (continued)

Verification code appendix: number_theory_lab.py

```
"""Exact educational algorithms and traces. No cryptographic deployment API."""
from math import gcd,isqrt
def extended(a,b):
    sa=-1 if a<0 else 1;sb=-1 if b<0 else 1;a=abs(a);b=abs(b)
    r0,r1=a,b;x0,x1=1,0;y0,y1=0,1;trace=[]
    while r1:
        q=r0//r1;trace.append((r0,r1,q,r0-q*r1))
        r0,r1=r1,r0-q*r1;x0,x1=x1,x0-q*x1;y0,y1=y1,y0-q*y1
    return r0,x0*sa,y0*sb,trace
def linear(a,b,n):
    if n<1:raise ValueError('modulus must be positive')
    g,u,_,_=extended(a,n)
    if b%g:return []
    step=n//g;x=(u*(b//g))%step
    return [x+k*step for k in range(g)]
def crt(pairs):
    x=0;m=1
    for a,n in pairs:
        if n<1:raise ValueError('positive moduli required')
        g,u,_,_=extended(m,n)
        if (a-x)%g:return None
        k=((a-x)//g*u)%(n//g);x+=m*k;m*=n//g;x%=m
    return x,m
def modular_power(a,b,n):
    if n<1 or b<0:raise ValueError('n>=1 and exponent>=0 required')
    a%=n;r=1%n;v=0;trace=[]
    for bit in (bin(b)[2:] if b else ''):
        before=r;square=r*r%n;r=square
        if bit=='1':r=r*a%n
        v=2*v+int(bit);trace.append((int(bit),v,before,square,r))
    return r,trace
def strong_pass(n,a):
    if n<5 or n%2==0:raise ValueError('odd n>=5 required')
    if not 1<=a<n:raise ValueError('base in1..n-1 required')
    d=n-1;s=0
    while d%2==0:s+=1;d//=2
    x=modular_power(a,d,n)[0];chain=[x]
    if x in (1,n-1):return True,chain
    for _ in range(s-1):
        x=x*x%n;chain.append(x)
        if x==n-1:return True,chain
    return False,chain
def probable_prime(n,bases):
    if n<2:return False
    if n in (2,3):return True
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Number-theoretic algorithms solutions (continued)

```
    if n%2==0:return False
    for a in bases:
        if not 2<=a<=n-2:raise ValueError('random test base must be in2..n-2')
        if not strong_pass(n,a)[0]:return False
    return True
def trial_factors(n):
    if n<2:raise ValueError('n>=2 required')
    ans=[];p=2
    while p*p<=n:
        e=0
        while n%p==0:n//=p;e+=1
        if e:ans.append((p,e))
        p+=1
    if n>1:ans.append((n,1))
    return ans
def liar_count_formula(n):
    if n<5 or n%2==0:raise ValueError('odd n>=5')
    d=n-1;s=0
    while d%2==0:s+=1;d//=2
    factors=trial_factors(n);ts=[];D=1
    for p,_ in factors:
        u=p-1;t=0
        while u%2==0:t+=1;u//=2
        ts.append(t);D*=gcd(d,u)
    tau=min(s,*ts);r=len(factors)
    return D*(1+sum(2**(j*r) for j in range(tau)))
def rho(n,x0=2,c=1,budget=10000):
    if n<=1:raise ValueError('n>1')
    if n==2:return None,[]
    if n%2==0:return 2,[]
    f=lambda x:(x*x+c)%n;x=y=x0%n;trace=[]
    for i in range(1,budget+1):
        x=f(x);y=f(f(y));d=gcd(abs(x-y),n);trace.append((i,x,y,abs(x-y),d))
        if 1<d<n:return d,trace
        if d==n:return None,trace
    return None,trace
if __name__=='__main__':
    assert rho(2)==(None,[])
    for candidate in (3,4,6,9,15,25,49,8051):
        factor,_=rho(candidate)
        assert factor is None or (1<factor<candidate and candidate%factor==0)

print('Bezout',extended(252,198));print('linear',linear(18,30,42));print('CRT',crt([(2,3),
(4,5),(1,7)]))

print('modexp',modular_power(5,13,11));print('MR341',strong_pass(341,2));print('RSA',pow(11,7
,187),pow(88,23,187));print('rho',rho(8051))
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Number-theoretic algorithms solutions (continued)

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study